

Parrainage et commissions — guide d'intégration mobile

Ce que le parrain voit, et ce que le filleul saisit. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.
Référence : issue #246. Complète le [guide des abonnements](#).

1. Ce qui change

Le code de parrainage servait jusqu'ici au **suivi** : savoir qui amène des utilisateurs. Il devient rémunérateur — quand un filleul souscrit un abonnement, le parrain reçoit un pourcentage du prix payé, **crédité dans son wallet**.

Deux surfaces côté application :

- un **écran « Mes parrainages »** : filleuls, commissions perçues ;
- un **champ code de parrainage** optionnel à la souscription, pour les

utilisateurs qui n'en avaient pas saisi à l'inscription.

Rien à faire pour déclencher le versement : il part du serveur quand l'abonnement est encaissé.

⚠ La commission est désactivée au lancement

Elle est livrée à **0 %**, **désactivée**. Un taux non arbitré qui verserait de l'argent réel dès le premier abonnement serait difficile à rattraper. L'administration l'active depuis le back-office.

Conséquence : `commissions.total` vaudra **0** pour tout le monde au début. Ce n'est pas une panne — prévoyez un état vide qui a du sens plutôt qu'un « 0 XOF » sec.

Authentification. Jeton JWT requis. **Scope pays** habituel.

2. Le code du parrain — GET /utilisateurs/code-parrainage

Inchangé, déjà en place :

```
{ "code_parrainage": "KLVCM1" }
```

C'est ce code que le parrain partage. Six caractères, majuscules.

3. Mes parrainages — GET /abonnements/mes-parrainages?country=benin

```
{
  "filleuls": [
    { "uuid": "d7bfb703-...", "nom": "T246", "prenom": "f2", "inscrit_le": "2026-09-06T08:32:25.802Z" },
    { "uuid": "6b3f9506-...", "nom": "T246", "prenom": "f1", "inscrit_le": "2026-09-06T08:32:24.601Z" }
  ],
  "nombre_filleuls": 2,
  "filleuls_abonnes": 2,
  "commissions": { "nombre": 2, "total": 400 }
}
```

champ	sens
filleuls	100 derniers inscrits, du plus récent au plus ancien
nombre_filleuls	taille de la liste renvoyée
filleuls_abonnes	filleuls dont l'abonnement a produit une commission
commissions.nombre	versements effectués
commissions.total	montant cumulé, dans la devise du wallet

L'email des filleuls n'est pas exposé — un parrain n'a pas à récupérer les adresses des gens qu'il a amenés. Affichez prénom et nom.

Le montant atterrit dans le wallet du parrain : les écrans wallet existants (solde, transactions, retrait) le montrent sans modification. Les transactions de commission portent `reward_source_type_code = "PARRAINAGE_ABONNEMENT"`.

4. Saisir un code à la souscription

POST /abonnements/souscrire accepte un `code_parrainage` optionnel :

```
{ "plan_uuid": "574612e9-...", "code_parrainage": "KLVCM1", "pays": "benin" }
```

La règle à comprendre

Le code n'est pris en compte que si l'utilisateur n'avait PAS déjà de parrain. La relation posée à l'inscription prime et n'est jamais réécrite : c'est une donnée d'acquisition, pas un champ modifiable.

situation de l'utilisateur	effet du <code>code_parrainage</code>
parrain défini à l'inscription	ignoré — le parrain d'origine garde la commission
aucun parrain	le code est résolu et le parrain rattaché
code inconnu	ignoré, la souscription réussit quand même
son propre code	ignoré (auto-parrainage refusé)

Ne présentez donc pas le champ à un utilisateur qui a déjà un parrain — il saisirait un code sans effet et croirait avoir crédité quelqu'un. `GET /abonnements/mon-abonnement` ne dit pas s'il en a un ; le plus simple est de n'afficher le champ que lorsque l'utilisateur n'a rien saisi à l'inscription, information dont l'application dispose déjà à ce moment-là.

Un code invalide **ne fait pas échouer la souscription**. C'est délibéré : bloquer un paiement pour une faute de frappe sur un champ facultatif serait absurde. Mais l'application ne recevra aucun signal d'erreur — ne promettez donc pas au filleul que son code « a été pris en compte ».

5. Quand la commission part

À l'**encaissement** de l'abonnement, pas à la souscription. Aujourd'hui c'est un administrateur qui confirme le paiement ; avec l'intégration du paiement en ligne, ce sera le webhook du prestataire.

```
souscription → abonnement EN_ATTENTE, parrain figé
               ↓ paiement encaissé
activation    → abonnement ACTIF + commission créditée au parrain
```

Le parrain est **figé à la souscription**. S'il change entre-temps, la commission va bien à celui qui était rattaché au moment de l'achat.

Un versement peut échouer sans conséquence pour le filleul : son abonnement s'active quoi qu'il arrive. Les commissions en souffrance sont rattrapées par l'administration.

Aucune notification spécifique n'est émise pour l'instant côté commission — le parrain la découvre sur son wallet ou sur l'écran « Mes parrainages ».

6. Ce que l'écran doit faire

Un état vide qui explique. Sans filleul : « Partagez votre code, gagnez sur chaque abonnement ». Avec des filleuls mais 0 commission : « Vos filleuls n'ont pas encore souscrit » — et non « 0 XOF », qui se lit comme une panne.

Rendre le code partageable. Bouton copier + partage système. C'est le seul geste qui fait vivre la fonctionnalité.

Ne pas promettre de montant. Le taux est réglable côté serveur et vaut 0 au lancement : n'affichez pas « gagnez 10 % » en dur.

Renvoyer vers le wallet. La commission y est visible avec le reste ; un lien évite de dupliquer un historique.

7. Exemple Flutter

```
class ParrainageApi {
  ParrainageApi(this._client, {required this.baseUrl, required this.token, required this.pays});
  final http.Client _client;
  final String baseUrl, token, pays;

  Map<String, String> get _entetes =>
    {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'};

  Future<Parrainages> mesParrainages() async {
    final r = await _client.get(
      Uri.parse('$baseUrl/abonnements/mes-parrainages?country=$pays'),
      headers: _entetes,
    );
    return Parrainages.fromJson(jsonDecode(r.body));
  }

  Future<String> monCode() async {
    final r = await _client.get(
      Uri.parse('$baseUrl/utilisateurs/code-parrainage?country=$pays'),
      headers: _entetes,
    );
    return (jsonDecode(r.body) as Map<String, dynamic>)['code_parrainage'] as String;
  }
}

class Parrainages {
  const Parrainages({
    required this.filleuls,
    required this.filleulsAbonnes,
    required this.nombreCommissions,
    required this.totalCommissions,
  });

  final List<Filleul> filleuls;
  final int filleulsAbonnes, nombreCommissions;
  final num totalCommissions;

  /// Distinguer « pas encore de filleul » de « filleuls sans abonnement » :
  /// les deux valent 0 XOF mais n'appellent pas le même message.
  bool get aucunFilleul => filleuls.isEmpty;
  bool get aucuneCommission => filleuls.isNotEmpty && nombreCommissions == 0;
}
```

Souscrire avec un code

```
Future<Abonnement> souscrire(String planUuid, {String? codeParrainage}) async {
  final r = await _client.post(
    Uri.parse('$baseUrl/abonnements/souscrire'),
    headers: _entetes,
    body: jsonEncode({
      'plan_uuid': planUuid,
      'pays': pays,
      // Ignoré côté serveur si l'utilisateur a déjà un parrain – ne l'annoncez
      // donc pas comme « pris en compte » à l'écran.
      if (codeParrainage != null && codeParrainage.isNotEmpty)
        'code_parrainage': codeParrainage.toUpperCase(),
    })),
  );
  if (r.statusCode ~/ 100 != 2) throw ApiException(messageErreur(jsonDecode(r.body)));
  return Abonnement.fromJson(jsonDecode(r.body));
}
```

8. Rappels

La commission est à 0 % et désactivée au lancement. N'affichez aucun taux en dur, et prévoyez l'état « aucune commission ».

Le code saisi à la souscription est ignoré si l'utilisateur a déjà un parrain. Ne montrez le champ que dans le cas contraire.

Un code invalide n'échoue pas. Aucune erreur ne remonte : ne confirmez pas au filleul que son code a fonctionné.

Le versement suit le paiement, pas la souscription. Un abonnement `EN_ATTENTE` n'a encore rien crédité.

L'email des filleuls n'est pas exposé. Prénom et nom seulement.

La commission arrive dans le wallet existant — pas de nouvel écran de solde à construire.